

Disciplina: Redes de Computadores

1º Semestre de 2026

Professor: Marcos Augusto Menezes Vieira (*mmvieira@dcc.ufmg.br*)

Monitor: Filipe Pirola Santos (*filipepirola@dcc.ufmg.br*)

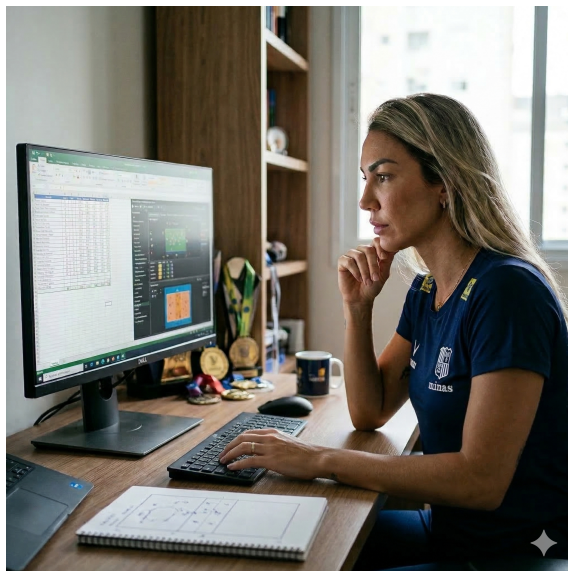
Data de entrega: 15/04/2026

Trabalho Prático 1: Operação Codebreaker

Após conquistar o mundo nas quadras e se tornar uma das atletas mais premiadas da história, a central Thaísa decidiu encarar um novo desafio: o universo da programação. Motivada por sua nova paixão, ela se inscreveu na MOP (Maratona Olímpica de Programação) para testar seus conhecimentos. Após superar diversas rodadas eliminatórias, Thaísa chegou à grande final contra sua eterna rival, a russa Gamova.

Contudo, em um movimento digno de um bloqueio inesperado, Gamova hackeou o terminal de Thaísa, instalando um sistema de bloqueio por código para impedir que a brasileira acesse o ambiente da competição. Sem conseguir logar, Thaísa recorreu a você, um talentoso estudante de Redes de Computadores da UFMG.

Sua missão é desenvolver um sistema **cliente-servidor** de quebra de códigos. O servidor (o computador hackeado) possui uma senha secreta, e o cliente (você ajudando a Thaísa) deve realizar tentativas para adivinhá-la. A cada tentativa, o servidor retornará dicas sobre a precisão dos caracteres, permitindo que a nossa bicampeã olímpica recupere seu acesso e conquiste mais uma medalha de ouro, desta vez nos bits e bytes.



Ajude a Thaísa a vencer a Olimpíada de Programação!

1 O Projeto

O objetivo deste trabalho é desenvolver um sistema de quebra de senhas (*Codebreaker*) baseado na mecânica de tentativa e erro com *feedback* posicional. O sistema deve seguir o modelo **cliente-servidor**, permitindo que o usuário (no papel de assistente técnico da Thaísa) tente descobrir o código secreto gerado ou armazenado pelo servidor (o terminal bloqueado pela Gamova).

1.1 Dinâmica da Competição

A "criptografia" do código ocorre através de rodadas de interação:

- O **servidor** inicializa o jogo carregando uma senha secreta (composta por uma sequência de 5 números).
- O **cliente** envia um palpite (*guess*) com a mesma quantidade de caracteres da senha original.
- A cada tentativa, o **servidor** processa a mensagem e retorna um *feedback* detalhado:
 - **Acertos Exatos:** Quantidade de números que estão na posição correta.
 - **Acertos de Caractere:** Quantidade de números que existem na senha, mas estão em posições erradas.
- O jogo termina quando o cliente acerta a sequência completa, desbloqueando o acesso da Thaísa e encerrando a conexão.

1.2 Requisitos Técnicos

Para garantir que o sistema resista às interferências da rede russa e siga os padrões do Departamento de Ciência da Computação da UFMG, as seguintes restrições devem ser seguidas:

- **Linguagem:** Implementação obrigatória em **C**, utilizando a interface POSIX de *sockets*. O uso de **C++** ou bibliotecas de alto nível é estritamente proibido.
- **Ambiente:** O sistema deve ser desenvolvido e testado exclusivamente para ambiente **Linux**. Bibliotecas específicas de outros sistemas operacionais (como *winsock.h*) não serão aceitas.
- **Rede:** A comunicação deve ser estabelecida via protocolo **TCP**, garantindo a entrega ordenada dos palpites. O sistema deve suportar tanto **IPv4** quanto **IPv6**.
- **Interoperabilidade:** As mensagens devem seguir um protocolo estrito (definido na Seção de Comunicação) para que o cliente de um aluno consiga desbloquear o servidor de outro.

1.3 Parâmetros de Inicialização

Os executáveis devem aceitar os seguintes argumentos via linha de comando para permitir testes automatizados:

Servidor (Terminal): `./server <protocolo> <porta> <senha>`

Onde `<protocolo>` pode ser `v4` ou `v6` e `<senha>` é a sequência de 5 dígitos numéricos que deve ser descoberta.

Cliente (Hacker): `./client <endereço_ip> <porta>`

Onde `<endereço_ip>` é o endereço do servidor hackeado.

2 A Quebra do Código

O servidor aguarda que o cliente envie um palpite para tentar descobrir a senha. O feedback é individual para cada uma das 5 posições, conforme as categorias da Tabela 1:

Tabela 1: Categorias de Feedback por Posição

Tipo	Status do Caractere	Significado
2	<i>Posição Correta</i>	O número daquela posição está correto.
1	<i>Caractere Presente</i>	O número existe na senha, mas em outra posição.
0	<i>Caractere Ausente</i>	O número não faz parte da senha.

O resultado da tentativa é mapeado para os seguintes estados do sistema:

- **1 (Acesso Concedido):** O cliente enviou os 5 números corretos nas posições exatas. Thaísa recupera o acesso e a conexão é encerrada com sucesso.

- **0 (Acesso Negado):** O palpite de 5 números está incorreto. O servidor retorna o vetor de dicas (Tabela 1) para orientar o próximo palpite.
- **-1 (Erro de Protocolo):** O cliente enviou um palpite com **tamanho diferente de 5 caracteres** ou utilizou caracteres fora do conjunto permitido.

3 Tratamento de Erros e Robustez

A resiliência do terminal é garantida pelo tratamento rigoroso de entradas inválidas, um requisito essencial para a estabilidade do sistema. Caso o cliente envie um palpite com tamanho diferente de 5 caracteres ou que contenha símbolos não numéricos, o servidor não deve encerrar a conexão abruptamente.

Em vez disso, o servidor deve descartar o pacote inválido, retornar o código de erro correspondente (Tipo -1) e aguardar um novo envio. Enquanto isso, o cliente deve tratar esse retorno e exibir na tela para o usuário a seguinte mensagem de alerta:

“Insira uma sequência válida!”

Dessa forma, tentativas inválidas não devem resetar o progresso da rodada atual, permitindo que a jogadora continue a tentativa de quebra do código sem penalidades de conexão.

4 Protocolo de Comunicação

4.1 Estrutura da Mensagem

A comunicação via *sockets* entre o cliente e o servidor será realizada através de uma estrutura de mensagem padronizada. Essa estrutura garante que os palpites numéricos e os *feedbacks* posicionais sejam interpretados corretamente, seguindo as normas de interoperabilidade do projeto.

Listing 1: Definição do Protocolo Operação Hacker

```
#define MSG_SIZE 128

typedef enum {
    MSG.START,      // Servidor solicita a senha de acesso
    MSG.GUESS,      // Cliente envia o palpite de 5 dígitos
    MSG.FEEDBACK,   // Servidor retorna as dicas (0, 1 ou 2)
    MSG.WIN,        // Servidor informa que o código foi quebrado
    MSG.ERROR,      // Servidor reporta erro de formato/tamanho
    MSG.EXIT        // Encerramento da conexão
} MessageType;

typedef struct {
    int type;                // Tipo da mensagem (MessageType)
    int guess[5];           // Vetor com os 5 dígitos enviados pelo cliente
    int feedback[5];        // Dicas retornadas pelo servidor (2, 1 ou 0)
    int attempts;           // Contador de tentativas realizadas
    int win_status;         // 1 para vitória, 0 para em jogo, -1 para erro
    char message[MSG_SIZE]; // Mensagem de texto para logs do terminal
} HackerMessage;
```

Observação: É obrigatório que a mensagem siga exatamente essa estrutura para garantir que o cliente de um aluno consiga invadir o servidor de outro.

4.2 Fluxo de Mensagens

O ciclo de vida da Operação Hacker segue a ordem rigorosa de requisição e resposta:

1. **Servidor → Cliente:** Envia `tipo = MSG_START` para inicializar as tentativas.

2. **Cliente → Servidor:** Envia `tipo = MSG_GUESS` com o vetor `guess[5]` preenchido com dígitos de 0 a 9.
3. **Servidor → Cliente:** Responde com `tipo = MSG_FEEDBACK`, preenchendo o vetor `feedback[5]` com as dicas e atualizando o contador de `attempts`.
4. **Servidor → Cliente:** Caso os 5 dígitos estejam na posição correta, envia `tipo = MSG_WIN` e encerra o desafio.

5 Execução

5.1 Execução do Servidor

Para inicializar o servidor, é necessário especificar o protocolo de rede e a porta de escuta. Por exemplo, para utilizar IPv4 na porta 51511, com a senha 54389, utilize o comando:

```
./server v4 51511 54389
```

5.2 Execução do Cliente

Para conectar o cliente a um servidor ativo, deve-se informar o endereço IP e a porta de destino. Por exemplo, para conectar-se ao servidor local (*localhost*) na porta 51511:

```
./client 127.0.0.1 51511
```

6 Exemplos de Uso

Para facilitar a implementação e garantir a interoperabilidade entre os sistemas, apresentamos um cenário de invasão via IPv4. Neste exemplo, o servidor é iniciado com a senha **54321**. O jogo termina quando a Thaísa acerta todos os dígitos.

Terminal do Cliente (Hacker)	Terminal do Servidor (Terminal)
	\$./server v4 51511 54321 Servidor iniciado em modo IPv4 na porta 51511.
\$./client 127.0.0.1 51511	Cliente Conectado
Insira seu palpite: > 92345 Dica: _ * 3 * * Tentativas realizadas: 1	
Insira seu palpite: > 543281 Insira uma sequência válida!	
Insira seu palpite: > 45328 Dica: * * 3 2 _ Tentativas realizadas: 2	
Insira seu palpite: > 54321 Acesso concedido! Thaísa recuperou o sistema!	Cliente Desconectado

7 Padronização da Saída (Interface)

Para possibilitar a correção automática e a interoperabilidade entre diferentes implementações, os programas devem seguir rigorosamente o padrão de exibição de texto descrito nesta seção. **Qualquer caractere extra, espaços adicionais ou mensagens personalizadas fora do padrão podem resultar em falha nos testes automatizados.**

7.1 Saída do Cliente (Hacker)

O cliente deve interagir com o usuário e exibir o feedback do servidor seguindo estas regras:

- **Prompt de Entrada:** Sempre que o programa solicitar um palpite, deve exibir a frase `Insira seu palpite:` (seguido de uma quebra de linha).
- **Exibição da Dica:** O feedback posicional deve ser impresso em uma única linha, precedido por `Dica:` . Para cada uma das 5 posições, utilize:
 - `<número>` para Posição Correta.
 - `*` para Caractere Presente.
 - `_` (underline) para Caractere Ausente.

Exemplo de saída para o palpite `32410` para a senha `39740`: `Dica: 3 _ * _ 0`

- **Contador de Tentativas:** Após a dica, o cliente deve imprimir: `Tentativas realizadas: N`, onde `N` é o número total de palpites enviados.
- **Mensagens de Estado:**
 - Erro de formato: `Insira uma sequência válida!`
 - Vitória: `Acesso concedido! Thaísa recuperou o sistema!`

7.2 Saída do Servidor (Terminal)

Embora o servidor opere em segundo plano, ele deve registrar eventos básicos para depuração:

- Ao iniciar: `Servidor iniciado em modo <IPv4/IPv6> na porta <porta>`
- Ao receber conexão: `Cliente conectado`
- Ao encerrar: `Cliente desconectado`

8 Informações sobre a Entrega

A entrega deve ser feita no formato ZIP, com o nome seguindo o seguinte padrão:

TP1_NOME_MATRICULA.zip

8.1 Documentação

Cada aluno deve entregar uma documentação em PDF de até **4 páginas**. A documentação deve discutir os desafios, dificuldades e imprevistos do projeto, bem como as soluções adotadas para os problemas. É imprescindível que vocês discutam todos os desafios que encontrarem, isso será muito importante para a avaliação.

8.2 Código

Além da documentação, cada aluno deve entregar o **código fonte em C** e um **Makefile** para a compilação do programa. Seu servidor será corrigido de forma semi-automática por uma bateria de testes. Para garantir o pleno funcionamento da correção, o código de vocês deve seguir as seguintes instruções:

- O Makefile deve compilar o cliente em um binário chamado “client” e o servidor em um binário chamado “server” dentro da raiz do projeto.
- Seu programa deve ser compilado com a execução do comando `make`, ou seja, sem a necessidade de parâmetros adicionais. Para isso, você precisa escrever um Makefile funcional

Procure escrever seu código de maneira clara, com comentários pontuais e bem identados. Isto facilita a correção do monitor e tem impacto positivo na avaliação.

9 Avaliação

O sistema será avaliado de forma automatizada, considerando o suporte aos protocolos IPv4 e IPv6. A distribuição da pontuação seguirá os critérios detalhados na Tabela 2.

Tabela 2: Distribuição de pontos por critério de avaliação.

Critério	Peso
Inicialização correta do servidor	10%
Conexão do cliente estabelecida com o servidor	15%
Funcionamento das jogadas (Cliente → Servidor)	10%
Funcionamento correto das dicas	10%
Contabilização de tentativas	10%
Resultado do encerramento do jogo	5%
Fechamento adequado de conexões	10%
Tratamento de erros e exceções	5%
Formatação e impressão de mensagens	5%
Documentação técnica	20%
Total	100%

Será adotada a média harmônica entre as notas da documentação e da execução, o que implica que a nota final será **zero** se uma das partes não for apresentada.

Atenção: Se não for possível compilar seu trabalho usando o comando make, sua nota final será automaticamente zero!

10 Política de Atrasos

Os trabalhos deverão ser entregues até as **23:59** do dia **15/04/2026** via Moodle. A partir desse horário, a nota final será penalizada de acordo com o atraso (em dias), seguindo a métrica:

$$Nota_{final} = Nota_{bruta} \times \left(1 - \frac{d}{3}\right)$$

onde d representa o número de dias de atraso.

Atenção: Após 3 dias de atraso, o trabalho não será mais aceito!

11 Detecção de Plágio

Sob nenhuma circunstância serão aceitos trabalhos com trechos de código gerados por inteligências artificiais. Espera-se que os alunos sejam capazes de desenvolver a solução sem copiar saídas de LLMs (*Large Language Models*). Portanto, qualquer sinal de plágio ou uso de código gerado artificialmente resultará em nota final **zero**.

Para garantir a integridade das avaliações, utilizaremos ferramentas de análise de código capazes de detectar "impressões digitais" e similaridades estruturais com alta precisão.

Além disso, vocês tem liberdade para conversar com os colegas sobre eventuais dúvidas e até testar os servidores dos outros, entretanto qualquer cópia de código é estritamente proibida e, se encontrada, na penalização da nota de todos os envolvidos.

Sejam criativos e, acima de tudo, originais!

12 Materiais de estudo

- Capítulo 2 e 3 do livro sobre programação com sockets disponibilizado no Moodle.
- Vídeo aula: "Programação com sockets, professor Ítalo Cunha"